



Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

Program design: Introduction

Copyright ©2026 by the 2026 staff of COS 110. All rights reserved.

Practical 1: Math and more Math

Release Date: 27-07-2026 at 06:00

Due Date: 31-07-2026 at 23:59

Late Deadline: 01-08-2026 at 00:59

Total Marks: 116

**Read the entire specification before starting
with the practical.**

Contents

1	General Instructions	3
2	Overview	4
2.1	Warning	4
2.2	Mark Breakdown	4
3	Your Task:	4
3.1	Flowchart	4
3.2	Permutations.h	5
3.2.1	Members	5
3.2.1.1	struct NegativeKError	5
3.2.1.2	struct EmptySetError	5
3.2.1.3	struct NullPointerError	5
3.2.1.4	struct Statistics	6
3.2.2	Functions	7
3.2.2.1	string intToString(int num)	7
3.2.2.2	void validateInput(char set[], int k, int n)	7
3.2.2.3	string printAllKLength(char set[], int k, int n)	7
3.3	BasicMath.h	8
3.3.1	Members	8
3.3.1.1	struct NegativeExponentError	8
3.3.1.2	struct ZeroDivisionError	8
3.3.1.3	struct InvalidRangeError	8
3.3.2	Functions	9
3.3.2.1	static string getIndent(int level)	9
3.3.2.2	static string intToString(int num)	9
3.3.2.3	string calculate_summation(int lower, int upper)	9
3.3.2.4	string calculate_power(int base, int exponent)	10
3.3.2.5	string calculate_gcd(int a, int b)	10
4	Memory Management	11
5	Testing	11
6	Implementation Details	12
7	Upload Checklist	12
8	Submission	13

1 General Instructions

- *Read the entire assignment thoroughly before you begin coding.*
- This assignment should be completed individually.
- Every submission will be inspected with the help of dedicated plagiarism detection software.
- Be ready to upload your assignment well before the deadline. There is a late deadline which is 1 hour after the initial deadline which has a penalty of 20% of your achieved mark. **No extensions will be granted.**
- If your code does not compile, you will be awarded a mark of 0. The output of your program will be primarily considered for marks, although internal structure may also be tested (eg. the presence/absence of certain functions or structure).
- Failure of your program to successfully exit will result in a mark of 0.
- Note that plagiarism is considered a very serious offence. Plagiarism will not be tolerated, and disciplinary action will be taken against offending students. Please refer to the University of Pretoria's plagiarism page at <https://portal.cs.up.ac.za/files/departamental-guide/>.
- Unless otherwise stated, the usage of non-C++98 features or additional libraries outside of those indicated in the assignment will **not** be allowed. Some of the appropriate files that you have submit will be overwritten during marking to ensure compliance to these requirements. **Please ensure you use C++98**
- All functions should be implemented in the corresponding `cpp` file. No inline implementation in the header file apart from the provided functions.
- The usage of ChatGPT and other AI-Related software to generate submitted code is strictly forbidden and will be considered as plagiarism.
- The use of this practical, in any form, by any company/business/organisation outside the University of Pretoria is strictly forbidden. This includes, but is not limited to, the use of the practical for discussion sessions, review sessions, marketing tools, or providing certain aspects as a free service.

2 Overview

This practical assignment focuses on implementing recursive algorithms with comprehensive error handling and execution tracing. Students will develop two main modules: a Permutations module that generates all possible strings of length k from a given character set, and a BasicMath module that implements recursive solutions for summation, exponentiation, and the Euclidean algorithm for finding the Greatest Common Divisor. Both modules require students to think recursively, handle edge cases gracefully, and provide detailed output showing the execution of the recursion that demonstrates their understanding of how recursion unfolds.

2.1 Warning

For this practical, you are only allowed to use recursion. The following rules must be adhered to throughout this practical. Failure to comply with any of these rules will result in a mark of 0. The words "for", "do", and "while" may not appear anywhere in any of the files you upload.

2.2 Mark Breakdown

The following table represents the total marks assigned to each task on FitchFork:

Task	Mark
Permutations	55
Summation	17
Power	14
GCD	19
Testing	11
Total	116

3 Your Task:

You are required to implement all of the functions laid out in this section.

3.1 Flowchart

Before you start coding, you need to plan out how each function needs to work. You will need to provide a single draw.io project with all the flowcharts. Each page in the draw.io project needs to represent a function. All functions need a flowchart representing their inner workings and must follow the correct flowchart notations. Each page's name should be the function's name. Upload this draw.io project onto the cs portal.

3.2 Permutations.h

The Permutations module implements a recursive algorithm to generate all possible strings of length k from a given set of characters, allowing repetition. This is also known as permutations with repetition or k -length strings from an n -element set.

The algorithm uses a backtracking approach where each position in the output string is filled with every character from the set, recursively exploring all combinations. The module includes comprehensive error handling for invalid inputs such as empty sets, negative k values, and null pointers.

3.2.1 Members

The module defines several structs for error handling. All members are public by design for simple error propagation. All of these are provided in the header file.

3.2.1.1 struct NegativeKError

- Thrown when $k < 0$ is passed as an argument
- Indicates that the requested permutation length is invalid
- Error message:

```
Caught exception: k must be non-negative\n
```

1

3.2.1.2 struct EmptySetError

- Thrown when the character set is empty
- Prevents recursion on an empty set
- Error message:

```
Caught exception: Set cannot be empty\n
```

1

3.2.1.3 struct NullPointerError

- Thrown when a NULL pointer is passed as the character set
- Ensures memory safety
- Error message:

```
Caught exception: Set pointer is NULL\n
```

1

3.2.1.4 struct Statistics

- Tracks performance metrics during execution.
- Contains:
 - `totalCombinations`: Total number of complete strings generated.
 - `maxDepth`: Maximum recursion depth reached during generation.
 - `allPrefixes`: Number of partial strings explored before reaching full length. Empty string doesn't count.
 - For example:

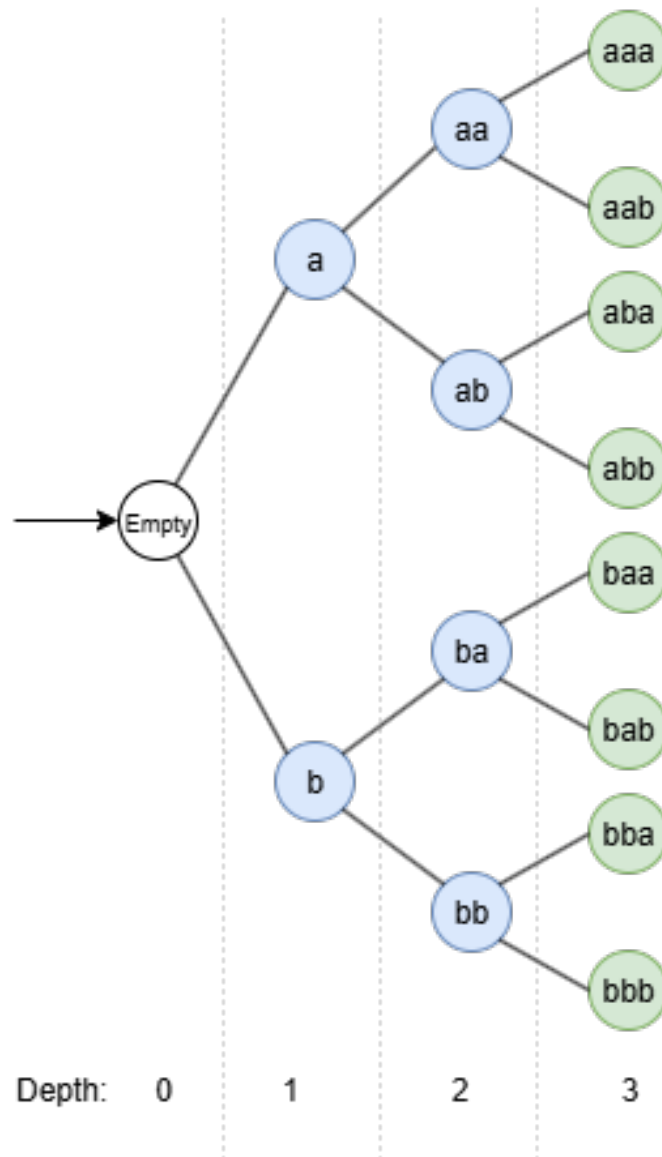


Figure 1: Permutations Example

Given that the Set $S = [a, b]$ and the length of the combination is 3.

Total combinations: 8 (Green)

Max recursion depth: 3

All prefixes: 6 (Blue)

3.2.2 Functions

The module provides both low-level recursive functions and high-level wrapper functions for user convenience. All functions are defined in `Permutations.h` and implemented in `Permutations.cpp`.

Also note that helper functions need to be implemented before functions that want to call the helper.

3.2.2.1 `string intToString(int num)`

- Helper function already provided.
- Converts an integer to a string using `stringstream`.
- Used for formatting the statistics in the output.

3.2.2.2 `void validateInput(char set[], int k, int n)`

- Validates all input parameters before the recursion starts
- Checks three conditions:
 1. If `set` is `NULL` $\rightarrow$ throw `NullPointerException`.
 2. If $n \leq 0 \rightarrow$ throw `EmptySetError`.
 3. If $k < 0 \rightarrow$ throw `NegativeKError`.

3.2.2.3 `string printAllKLength(char set[], int k, int n)`

- Creates and initializes a `Statistics` struct with all values set to 0.
- Generates all permutations by building strings character by character. Appends a blank line after each permutation.
- Appends a "[Statistics]" section containing:
 - Total combinations generated.
 - Maximum recursion depth reached.
 - Number of all prefixes explored.
- Returns the complete formatted string containing all permutations and statistics.
- To be able to implement this function, you will need a helper function.
- You should be checking whether an exception should be thrown, and if that's the case, it should be thrown and caught by this function. If the exception is caught, it should return a string containing the exception message.
- An example of what the output should look like is provided in the `out.txt` file.

3.3 BasicMath.h

The BasicMath module implements recursive solutions for fundamental mathematical operations: summation, exponentiation, and the Euclidean algorithm for finding the Greatest Common Divisor (GCD). Each operation is implemented recursively with comprehensive error handling and detailed execution tracing.

This module serves as an introduction to recursive thinking, demonstrating how complex mathematical problems can be broken down into smaller, self-similar subproblems. You will implement recursive functions that not only compute results but also trace their execution step by step, building an understanding of how recursion unfolds.

3.3.1 Members

The module defines several structs for error handling. All members are public by design for simple error propagation. All of these are provided in the header file.

3.3.1.1 struct NegativeExponentError

- Thrown when a negative exponent is passed to the power function.
- Contains an error message to display to the user.

```
Caught exception: Negative exponent not supported\n
```

1

3.3.1.2 struct ZeroDivisionError

- Thrown when attempting to divide by zero (e.g., GCD(0,0)).
- Contains an error message to display to the user.

```
Caught exception: Both numbers cannot be zero\n
```

1

3.3.1.3 struct InvalidRangeError

- Thrown when summation has an invalid range.
- The two cases are that the lower bound can't be greater than the upper bound or the range is too large.
- The max range is 10000.
- Contains an error message to display to the user.

```
Caught exception: Lower bound cannot be greater than upper bound\n
```

1

```
Caught exception: Range too large (max 10000)\n
```

1

3.3.2 Functions

The module provides wrapper functions that return formatted strings containing execution traces and results. You need to implement the recursive logic behind each operation. All functions are defined in `BasicMath.h` and implemented in `BasicMath.cpp`.

An example of how the output should look like is provided in the `out.txt` file. Also note that helper functions need to be implemented before functions that want to call the helper.

3.3.2.1 `static string getIndent(int level)`

- Helper function already provided.
- Generates an indentation string for tracing output.
- Returns a string of `level` spaces.
- Used to show recursion depth in output.

3.3.2.2 `static string intToString(int num)`

- Helper function already provided.
- Converts an integer to a string using `stringstream`.
- Used for formatting output.

3.3.2.3 `string calculate_summation(int lower, int upper)`

- Computes the sum of all integers from `lower` to `upper` inclusive. For example:
`calculate_summation(3,5)` should calculate $3 + 4 + 5 = 12$
- Returns a formatted string showing:
 - Each recursive call with proper indentation.
 - The value returned at each step.
 - The final result.
- Proper indentation should be done using the helper function.
- You should be checking whether an exception should be thrown, and if that's the case, it should be thrown and caught by this function. If the exception is caught, it should return a string containing the exception message.

3.3.2.4 string calculate_power(int base, int exponent)

- Computes **base** raised to the power of **exponent**.
- Returns a formatted string showing:
 - Each recursive call with proper indentation.
 - The multiplication at each step.
 - The final result.
- Error validation for negative exponents.
- Execution tracing showing multiplication steps.
- Proper indentation should be done using the helper function.
- You should be checking whether an exception should be thrown, and if that's the case, it should be thrown and caught by this function. If the exception is caught, it should return a string containing the exception message.

3.3.2.5 string calculate_gcd(int a, int b)

- Computes the Greatest Common Divisor of **a** and **b** using the Euclidean algorithm. This is an iterative version of the algorithm; convert it to a recursive one.

Algorithm 3.1 Euclidean algorithm for GCD given whole numbers **a** and **b**

```
if a is negative then
    Make a positive.
end if
if b is negative then
    Make b positive.
end if
while b is not 0 do
    Remainder := a Mod b
    Replace a with b
    Replace b with Remainder
end while
a is the GCD
```

- For example:
 - Calling gcd(20, 512)
 - Calling gcd(512, 20)
 - Calling gcd(20, 12)
 - Calling gcd(12, 8)
 - Calling gcd(8, 4)
 - Calling gcd(4, 0)

Since b is 0, a is the gcd of 20 and 512.

Result: 4

- Returns a formatted string showing:
 - Each recursive call with proper indentation.
 - The modulo operation at each step.
 - The final result.
- Error validation for zero inputs.
- Execution tracing showing each step of the algorithm.
- Proper indentation should be done using the helper function.
- You should be checking whether an exception should be thrown, and if that's the case, it should be thrown and caught by this function. If the exception is caught, it should return a string containing the exception message.

4 Memory Management

As memory management is a core part of C++ programming, each task on FitchFork will allocate approximately 10% of the marks to memory management. The following command is used:

```
valgrind --leak-check=full ./main
```

1

Please ensure, at all times, that your code *correctly* de-allocates *all* the memory that was allocated.

5 Testing

As testing is a vital skill that all software developers need to know and be able to perform daily, 10% of the assignment marks will be allocated to your testing skills. To do this, you will need to submit a testing main (inside the `main.cpp` file) that will be used to test an Instructor Provided solution. You may add any helper functions to the `main.cpp` file to aid your testing. In order to determine the coverage of your testing the gcov¹ tool, specifically the following version *gcov (Debian 8.3.0-6) 8.3.0*, will be used. The following set of commands will be used to run gcov:

```
g++ --coverage *.cpp -o main
./main
gcov -f -m -r -j ${files}
```

1

2

3

This will generate output which we will use to determine your testing coverage. The following coverage ratio will be used:

$$\frac{\text{number of lines executed}}{\text{number of source code lines}}$$

¹For more information on gcov please see <https://gcc.gnu.org/onlinedocs/gcc/Gcov.html>

We will scale this ration based on class size.

The mark you will receive for the testing coverage task is determined using Table 1:

Coverage ratio range	% of testing mark
0%-5%	0%
5%-20%	20%
20%-40%	40%
40%-60%	60%
60%-80%	80%
80%-100%	100%

Table 1: Mark assignment for testing

Note the top boundary for the Coverage ratio range is not inclusive except for 100%. Also, note that only the functions stipulated in this specification will be considered to determine your testing mark. Remember that your main will be testing the Instructor Provided code and as such, it can only be assumed that the functions outlined in this specification are defined and implemented.

As you will be receiving marks for your testing main, plagiarism detection will also be performed on the testing main.

6 Implementation Details

- You must implement the functions in the header files exactly as stipulated in this specification. Failure to do so will result in compilation errors on FitchFork.
- You may only use **c++98**.
- You may only utilise the specified libraries. Failure to do so will result in compilation errors on FitchFork.
- You may only use the following libraries:
 - string
 - iostream
 - sstream
- You are supplied with a **trivial** main demonstrating the basic functionality of this assessment.

7 Upload Checklist

The following c++ files should be in a zip archive named uXXXXXXXXX.zip where XXXXXXXXX is your student number:

- `Permutations.h`
- `Permutations.cpp`
- `BasicMath.h`
- `BasicMath.cpp`
- `main.cpp`
- Any textfiles used by your `main.cpp`

The files should be in the root directory of your zip file. In other words, when you open your zip file you should immediately see your files. They should not be inside another folder.

8 Submission

You need to submit your source file(s), as a single archive, on the FitchFork website (<https://ff.cs.up.ac.za/>). All methods need to be implemented (or at least stubbed) before submission. Your code should be able to be compiled with the following command:

```
g++ -std=c++98 -Werror -Wall *.cpp -o main
```

1

and run with the following command:

```
./main
```

1

Remember your `h` file will be overwritten, so ensure you do not alter the provided `h` files.

You have 10 submissions, and your best mark will be your final mark. Upload your archive to the Practical 1 slot on the FitchFork website. If you submit after the deadline but before the late deadline, a 20% mark deduction will be applied. **No submissions after the late deadline will be accepted!**